



Školska godina 2025/26

PREDMET:

CS320 – Operativni sistemi

Projektni zadatak

UPRAVLJANJE PROCESIMA U KLASITER OKRUŽENJU

Ime i prezime: **Luka Golubović**

Smer: **Softversko Inženjerstvo**

Broj indeksa: **6356**

Datum izrade: **25.07.2026.**

Asistent: **Goran Stamenović**

Upravljanje procesima u klaster okruženju

Sadržaj

- [Sažetak](#)
- [Uvod](#)
- [Cilj rada](#)
- [Metodologija](#)
- [Razrada](#)
 - o [Zahtevi i granica projektnog rešenja](#)
 - o [Procesi i raspoređivanje](#)
 - o [Klaster kao skup ograničenih čvorova](#)
 - o [Arhitektura aplikacije](#)
 - o [Domenski model procesa](#)
 - o [Politike raspoređivanja](#)
 - o [Jedan takt simulacije](#)
 - o [Kvar čvora i ponovno raspoređivanje](#)
 - o [REST API i komandni alati](#)
 - o [Konfiguracija i isporuka](#)
 - o [Metrike](#)
 - o [Eksperimentalni scenariji](#)
 - o [Rezultati](#)
 - o [Tumačenje balansiranog i heterogenog opterećenja](#)
 - o [Tumačenje preopterećenja i prioriteta](#)
 - o [Tumačenje kvara čvora](#)
 - o [Ponovljivost i funkcionalna provera](#)
 - o [Testiranje](#)
 - o [Korektnost i složenost](#)
 - o [Bezbednosne i operativne granice](#)
 - o [Ograničenja i dalji rad](#)
- [Zaključak](#)
- [Spisak literature](#)

Sažetak

U radu je prikazana Go aplikacija koja simulira upravljanje procesima nad klasterom računarskih čvorova. Svaki čvor ima ograničen CPU i memorijski kapacitet, dok proces definiše zahteve za resursima, trajanje, prioritet i politiku ponovnog pokretanja. Sistem vodi životni ciklus procesa, bira čvor prema jednoj od tri politike raspoređivanja i proverava da nijedna odluka ne prekorači kapacitet.

Implementacija obuhvata deterministički simulator, REST servis i komandni klijent. Jezgro je podeljeno na domenski model, kontroler klastera, raspoređivače, izvršni engine, metrike i pomoćne skladišne operacije. Podržani su Round Robin, Least Loaded i Priority-Aware algoritmi. Kontroler obrađuje ručne i zakazane kvarove, heartbeat rokove, ograničene restart pokušaje, ponovno raspoređivanje, simulirani I/O, pauziranje i prekid procesa.

Studija slučaja koristi šest verzionisanih scenarija. Sedam referentnih pokretanja obuhvata balansirano, heterogeno, preopterećeno i prioritarno opterećenje, kao i kvar čvora. U svim referentnim pokretanjima završeno je 100% prijavljenih procesa. Balansirano opterećenje završava se za 9 taktova sa oba poređena raspoređivača, ali Least Loaded smanjuje srednju standardnu devijaciju opterećenja sa 0,1393 na 0,0987. Preopterećen scenario dostiže prosečnu CPU iskorišćenost od 89,10%, uz 82 odlaganja raspoređivanja i prosečno čekanje od 11,25 taktova. Scenario kvara završava pet procesa za 16 taktova, sa dva restarta i dva ponovna raspoređivanja.

Uvod

Proces je program u izvršavanju sa sopstvenim stanjem, resursima i tokom obrade. Operativni sistem održava podatke o procesu, određuje kada može da se izvršava i oslobađa resurse kada se proces završi. Kada više procesa traži ograničen procesor ili memoriju, odluke raspoređivača utiču na vreme čekanja, propusnost i iskorišćenost sistema (Tanenbaum i Bos 2015.; Silberschatz i dr. 2014.).

Klaster proširuje isti problem na skup računara. Odluka više nije ograničena na izbor sledećeg procesa za jedan procesor. Potrebno je izabrati čvor koji je dostupan i ima dovoljno resursa, a zatim pratiti promene njegovog stanja. Kvar jednog čvora pogađa sve procese koji su mu dodeljeni. Kontrolni sloj tada mora da utvrdi koji procesi smeju ponovo da se pokrenu i gde za njih postoji slobodan kapacitet.

Ovakav model postoji u sistemima za orkestraciju kontejnera. Kubernetes raspoređivanje, na primer, odvaja proveru izvodljivosti od izbora čvora: kandidati se filtriraju, ocenjuju i zatim vezuju za opterećenje (Kubernetes Documentation 2025.). Aplikacija iz ovog rada nije Kubernetes zamena. Ona isti problem svodi na dovoljno mali model da se svaka odluka može ponoviti i objasniti.

Simulacija koristi virtuelne taktove. CPU i memorija su celobrojne jedinice, a proces ne izvršava korisnički program. Takva granica uklanja mrežnu i hardversku neizvesnost. Ostaju pitanja koja su bitna za temu: kako predstaviti stanje procesa, kako odabrati čvor, kako sprečiti prekomernu alokaciju i kako obraditi kvar.

Teorijski deo je vezan direktno za odluke u kodu, dok najveći deo dokumenta opisuje model, algoritme, interfejs, testove i rezultate ponovljivih scenarija.

Cilj rada

Cilj je da se napravi proverljiva simulacija upravljanja procesima u klasteru. Model mora da pokaže šta se dešava od prijave procesa do završnog stanja, uključujući nedostatak kapaciteta i kvar čvora.

Iz tog cilja izvedeni su sledeći zadaci:

1. predstaviti čvorove, procese, događaje i stanje simulacije jasnim domenskim tipovima;
2. implementirati tri raspoređivača sa različitim kriterijumima izbora;
3. održati tačne CPU i memorijske alokacije tokom svake promene stanja;
4. omogućiti determinističko izvršavanje scenarija i rad u realnom vremenu;
5. izložiti funkcije kroz scenario runner, REST API i komandni klijent;
6. prikupiti metrike koje omogućavaju poređenje raspoređivača i opterećenja;
7. proveriti ponašanje automatizovanim testovima, race detektorom i build proverom.

Sistem je obrazovni simulator. Ne pokreće udaljene izvršne fajlove, ne obavlja migraciju memorijske slike procesa i ne koordinira više kontrolnih čvorova. Takav opseg omogućava da se raspoređivanje, promene stanja i oporavak ispitaju u ponovljivim uslovima.

Metodologija

Rad je izrađen iz dva skupa izvora. Teorijske tvrdnje zasnovane su na literaturi o procesima, raspoređivanju i klaster raspoređivačima. Tehničke tvrdnje proverene su prema završnom Go kodu, testovima, scenarijima i verzionisanim rezultatima. Time je izbegnuto opisivanje planirane funkcije kao da već postoji.

Razvoj modela krenuo je od stanja procesa i ograničenja resursa. Posle domenskih tipova uveden je interfejs raspoređivača, a zatim kontroler koji jedini menja aktivno stanje. Engine poziva kontroler u diskretnim taktovima ili preko vremenskog brojača. REST sloj i komandni alati koriste iste operacije, pa poslovna pravila nisu duplirana.

Za eksperimentalni deo korišćeni su JSON scenariji sa fiksnim seed-om 42. Svaki scenario sadrži naziv, raspoređivač, maksimalan broj taktova, čvorove, procese i opcione kvarove. Rezultat se izvozi u JSON ili CSV. Referentna tabela napravljena je iz verzionisanog CSV izveštaja, a komande za ponavljanje navedene su uz analizu.

Provere su sprovedene na više nivoa. Domenski testovi proveravaju validaciju i dozvoljene prelaze stanja. Testovi raspoređivača proveravaju izbor čvora i redosled procesa. Kontroler i engine testovi pokrivaju kvarove, rollback, konkurentni pristup, završetak bez napretka i determinističko ponavljanje. HTTP testovi koriste httptest, dok testovi izvršnih programa proveravaju CLI i konfiguraciju.

Razrada

Zahtevi i granica projektnog rešenja

Predmet rada je simulacija upravljanja procesima u klaster okruženju. Takva aplikacija mora da ima model procesa, model klastera, odluku o raspoređivanju i vidljiv rezultat.

Tabela 1 povezuje zahtev teme sa završnom implementacijom. Putanje su relativne u odnosu na koren izvornog repozitorijuma.

Zahtev	Realizacija	Dokaz u projektu
Izvršna aplikacija	REST servis, komandni klijent i scenario runner	cmd/server, cmd/clusterctl, cmd/simulator
Model procesa	Stanja, resursi, trajanje, prioritet i restart politika	internal/domain/process.go
Model klastera	Čvorovi, statusi, kapacitet i trenutno zauzeće	internal/domain/node.go
Raspoređivanje	Round Robin, Least Loaded i Priority-Aware	internal/scheduler/
Tok simulacije	Deterministički taktovi i opciono real-time ticker	internal/engine/
Kvar i oporavak	Kvar čvora/procesa, heartbeat i ograničeni restart	internal/cluster/recovery.go
Studija slučaja	Šest JSON scenarija i sedam referentnih rezultata	scenarios/, docs/results/
Provera rada	Paketni testovi, race detector, vet i build	*_test.go, Makefile, CI workflow

Aplikacija koristi samo Go standardnu biblioteku. Nema baze podataka ni spoljnog message brokera. Aktivno stanje se čuva u memoriji, dok izvoz rezultata i pomoćni snapshot format koriste JSON ili CSV. Takav izbor pojednostavljuje pokretanje i zadržava fokus na upravljanju procesima.

Procesi i raspoređivanje

Klasični model procesa obuhvata identitet, stanje izvršavanja, programski kontekst i resurse. U operativnom sistemu proces prelazi između spremnog, aktivnog, blokiranog i završnog stanja. Raspoređivač bira kada će proces dobiti procesor, dok mehanizmi sinhronizacije i I/O menjaju njegovu dostupnost (Tanenbaum i Bos 2015.).

Kriterijumi raspoređivanja zavise od namene sistema. Propusnost meri broj završenih poslova u vremenu. Vreme čekanja meri koliko dugo proces provodi spreman bez izvršavanja. Vreme obrta obuhvata period od prijave do završetka. Pravednost sprečava da pojedini proces čeka bez granice. Nijedan algoritam ne optimizuje sve kriterijume u svakoj raspodeli opterećenja (Silberschatz i dr. 2014.).

U klaster modelu proces je jedinica posla. Raspoređivač bira čvor, a ne jezgro procesora. Vremenski kvant postoji u ulaznom modelu i validira se, ali završna verzija ne vrši preempciju na isteku kvanta. Round Robin u ovom projektu kruži kroz čvorove prilikom smeštanja procesa. Ta razlika je bitna za tačno tumačenje rezultata.

Klaster kao skup ograničenih čvorova

Čvor ima CPU kapacitet, memorijski kapacitet, trenutno zauzeće i status. Novi proces može da se smesti samo na ONLINE čvor. Provera CanFit zahteva da slobodan CPU i memorija budu najmanje jednaki zahtevima procesa. Odluka se odbija čim jedan resurs nije dovoljan.

Status čvora može biti ONLINE, DRAINING, OFFLINE, FAILED ili RECOVERING. DRAINING čvor više ne prima nove procese jer raspoređivač prihvata samo ONLINE čvorove. Prelazak na OFFLINE dozvoljen je kada nema aktivnih procesa. Kvar odmah postavlja FAILED, uklanja alokacije i obrađuje pogođene procese.

Tabela 2 poredi klasično CPU raspoređivanje i model primenjen u simulatoru.

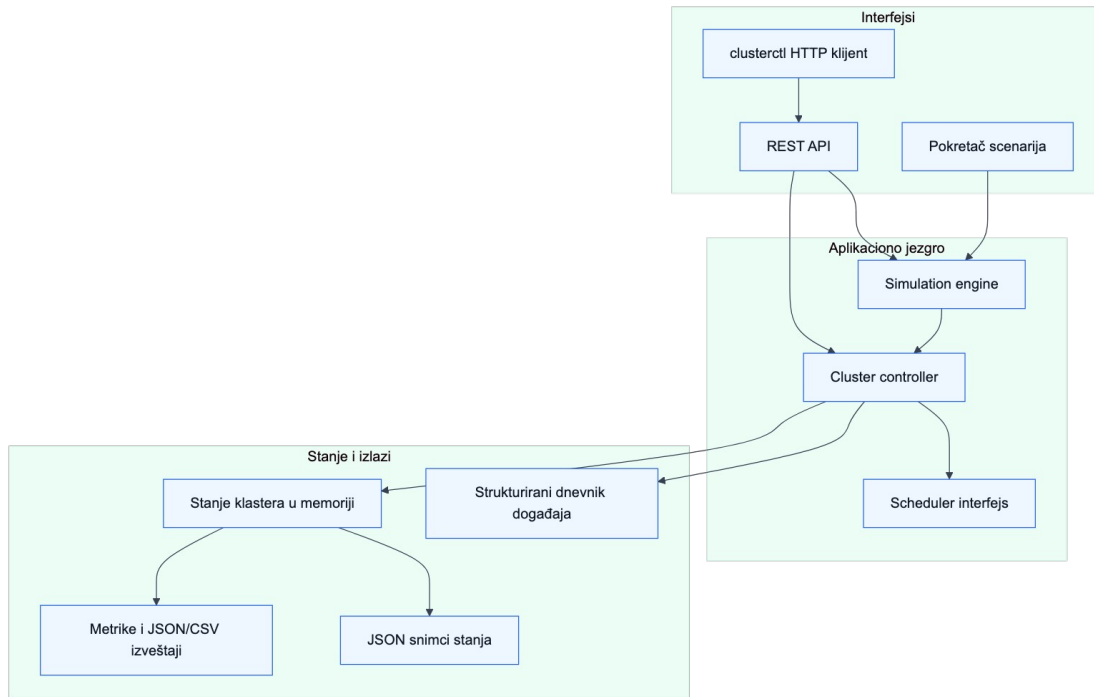
Osobina	CPU raspoređivanje	Klaster raspoređivanje u aplikaciji
Kandidat	Proces ili nit	Proces kao jedinica posla
Odredište	CPU jezgro ili red izvršavanja	Čvor sa CPU i memorijskim kapacitetom
Ograničenja	prioritet, kvant, stanje procesa	status čvora, CPU, memorija i politika
Kvar resursa	prekid ili kvar lokalnog sistema	kvar čvora i obrada svih njegovih procesa
Metrike	čekanje, odziv, obrt, propusnost	čekanje, obrt, propusnost, iskorišćenost i balans

Kubernetes Scheduling Framework takođe razdvaja proveru uslova i ocenjivanje kandidata (Kubernetes Documentation 2025.). Simulator koristi jednostavniju verziju tog obrasca. Metoda CanFit filtrira čvorove, a konkretni raspoređivač bira jednog od preostalih kandidata.

Arhitektura aplikacije

Arhitektura ima tri ulazna interfejsa i jedno aplikaciono jezgro. `cmd/simulator` učitava scenario i pokreće ga do završetka. `cmd/server` izlaže HTTP operacije. `cmd/clusterctl` je klijent za isti HTTP ugovor. Sve tri putanje na kraju koriste kontroler klastera i engine.

Slika 1 prikazuje glavne komponente i smer zavisnosti. Interfejsi ne menjaju stanje direktno. Kontroler poseduje aktivno stanje, raspoređivač dobija nepromenljive snimke čvorova, a metrike se grade iz duboke kopije stanja.



Slika 1. Arhitektura aplikacije

Tabela 3 opisuje pakete koji čine završnu strukturu.

Paket	Odgovornost
internal/domain	Proces, čvor, scenario, događaj, stanja i validacija
internal/scheduler	Interfejs i tri politike raspoređivanja
internal/cluster	Kontroler, životni ciklus, kvarovi, scenario i invarijante
internal/engine	Determinističko i real-time izvršavanje
internal/metrics	Izračunavanje i JSON/CSV izvoz izveštaja
internal/store	Izolovana memorijska kopija i JSON snapshot pomoćne funkcije
internal/transport/http	REST rute, validacija zahteva i mapiranje grešaka
internal/logging	Strukturirano JSON logovanje servera

Kontroler koristi `sync.RWMutex`. Operacije koje menjaju stanje uzimaju ekskluzivnu bravu, dok čitanje snimka i lista koristi `read` bravu. Metode za čitanje vraćaju kopije. Pozivalac zato ne može da menja mape, slice vrednosti ili pokazivače koji pripadaju aktivnom stanju.

Store paket pruža dodatnu izolovanu memorijsku kopiju i serijalizaciju snimka. On nije trajna baza i nije povezan kao obavezni runtime sloj servera. Njegova uloga je da proveri ugovor kopiranja i format stanja koji se kasnije može povezati sa fajlom ili bazom.

Domenski model procesa

Proces čuva identifikator, naziv, stanje, prioritet, CPU i memorijski zahtev, ukupno i preostalo trajanje, vremenski kvant, odabrani čvor i podatke o restartovanju. Uz to se beleže takt prijave, prvi početak, završetak i zbirno čekanje.

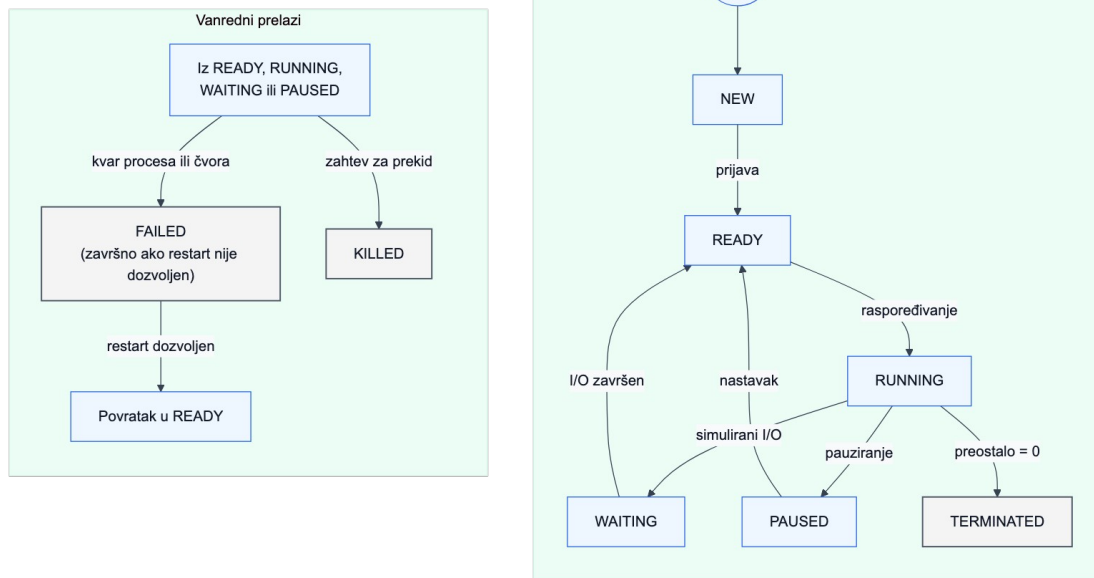
Tabela 4 navodi polja koja najviše utiču na izvršavanje.

Polje	Značenje u simulaciji
priority	Veća vrednost ima prednost kod Priority-Aware reda
cpuRequest	Broj CPU jedinica rezervisanih tokom RUNNING stanja
memoryRequestMB	Memorija rezervisana na čvoru
totalTicks	Trajanje jednog pokušaja procesa
remainingTicks	Broj preostalih taktova trenutnog pokušaja
timeQuantum	Validirano svojstvo modela; preempcija još nije implementirana
restartPolicy	NEVER, ON_FAILURE ili ALWAYS vrednost
maxRestarts	Gornja granica ponovnih pokretanja posle kvara
submitAtTick	Takt u kome proces postaje vidljiv kontroleru

Konstruktor postavlja stanje NEW, a prijava ga prevodi u READY. Raspoređivanje prelazi u RUNNING i rezerviše resurse. Simulirani I/O prelazi iz RUNNING u WAITING, pri čemu se resursi oslobađaju. Operacija wake vraća proces u READY. Pauza koristi zasebno stanje PAUSED, a resume ga takođe vraća u red spremnih procesa.

Završna stanja su TERMINATED, FAILED i KILLED. Proces postaje TERMINATED kada preostalo trajanje dođe do nule. Kvar vodi u FAILED; ako politika i brojač dozvoljavaju novi pokušaj, proces se odmah vraća u READY. Trenutna implementacija tada postavlja remainingTicks na punu vrednost totalTicks. To je restart od početka, a ne nastavak sa checkpoint-a.

Slika 2 prikazuje dozvoljene prelaze. Kontroler odbija operaciju koja ne odgovara trenutnom stanju. Na primer, samo RUNNING proces može da se pauzira ili pređe u simulirano I/O čekanje.



Slika 2. Životni ciklus procesa

Validacija zahteva neprazne identifikatore i nazive, pozitivan CPU, memoriju, trajanje i kvant, kao i nenegativan broj restart pokušaja. Scenario dodatno proverava duplikate, zakazane prijave posle maxTicks i kvarove koji upućuju na nepostojeći proces ili čvor.

Politike raspoređivanja

Svi raspoređivači implementiraju metod `SelectNode(process, nodes)`. Metod dobija vrednosti `NodeSnapshot`, bez pokazivača na aktivne čvorove. Time je odluka odvojena od alokacije. Kontroler prvo pita za čvor, a zatim sam proverava i upisuje resurse.

Round Robin sortira čvorove po ID-u, pamti kursor i kruži od sledeće pozicije. Čvor koji nije ONLINE ili nema kapacitet preskače se. Kursor je zaštićen mutex-om jer isti raspoređivač može biti pozvan iz konkurentnih ulaza. Ako nijedan čvor ne odgovara, proces ostaje u READY stanju i beleži se odlaganje.

Least Loaded računa:

$$\text{score} = 0,5 * (\text{zauzeti CPU} / \text{CPU kapacitet}) + 0,5 * (\text{zauzeta memorija} / \text{memorijski kapacitet})$$

Izabran je kandidat sa najmanjom ocenom pre smeštanja procesa. Ako dve ocene imaju istu vrednost, bira se leksikografski manji ID. Ovo pravilo uklanja zavisnost od redosleda iteracije kroz Go mapu.

Priority-Aware menja redosled READY procesa, a za izbor čvora koristi Least Loaded. Efektivni prioritet je osnovni prioritet uvećan za jedan poen na svakih pet taktova provedenih u redu. Veća vrednost ide prva. Kod izjednačenja prednost ima ranija prijava, a zatim manji ID. Aging smanjuje rizik da proces niskog prioriteta čeka bez granice.

Tabela 5 daje sažeto poređenje.

Raspoređivač	Redosled procesa	Izbor čvora	Tipična namena
Round Robin	prijava, zatim ID	kružni izbor dostupnih čvorova	jednostavna i ponovljiva raspodela
Least Loaded	prijava, zatim ID	najmanja kombinovana CPU/memorijska popunjenost	ravnomernije opterećenje
Priority-Aware	efektivni prioritet, prijava, ID	Least Loaded	hitni procesi uz aging

Round Robin ne garantuje najbolji balans kada čvorovi imaju različite kapacitete. Least Loaded ne uzima u obzir očekivano trajanje procesa. Priority-Aware može da odloži procese niskog prioriteta dok je kapacitet pun, iako aging postepeno popravlja njihov položaj. Zbog tih razlika rezultati se porede nad istim ulazom.

Jedan takt simulacije

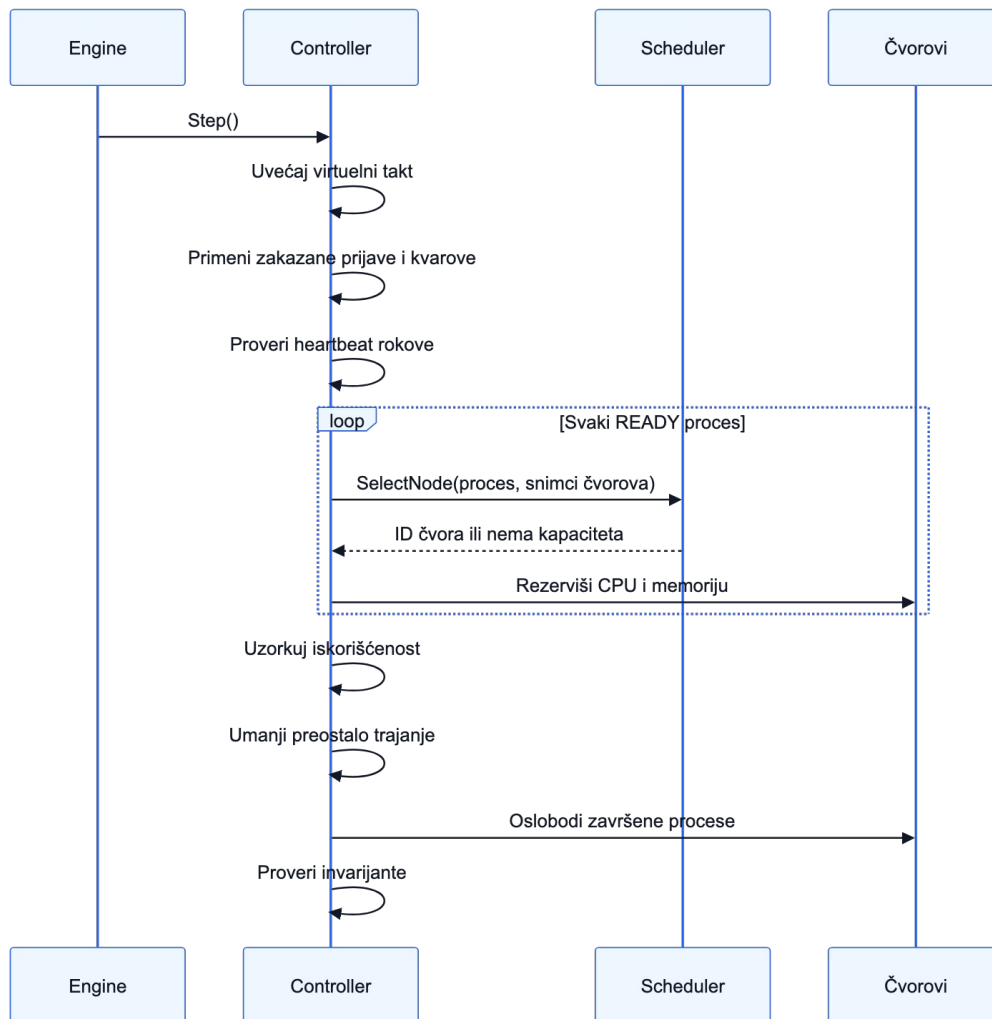
Engine može da pozove jedan ili više determinističkih koraka. Može i da pokrene goroutine sa `time.Ticker` petljom za rad u realnom vremenu. `Pause` i `Close` otkazuju petlju preko `context.Context` i čekaju njen završetak. Upotreba konteksta prati standardni Go obrazac za prenos otkazivanja između goroutine-a (Ajmani 2014.).

Kontroler u svakom koraku radi sledeće:

1. čuva kopiju stanja potrebnu za rollback;
2. uvećava virtuelni takt;
3. prijavljuje procese i primenjuje kvarove zakazane za taj takt;
4. proverava heartbeat rokove;
5. sortira i raspoređuje READY procese;
6. uzorkuje CPU, memoriju i odstupanje opterećenja;
7. umanjuje `remainingTicks` aktivnim procesima;
8. završava procese čije je trajanje došlo do nule;
9. određuje da li je simulacija završena;
10. proverava invarijante.

Ako zakazani događaj ili provera invarijante vrati grešku, kontroler vraća stanje, listu događaja, brojač sekvence i preostale zakazane stavke na vrednosti pre takta. Takt je transakcioni u granicama memorijskog modela.

Slika 3 prikazuje isti redosled i granicu između engine-a, kontrolera, raspoređivača, stanja i metrika.



Slika 3. Sekvenca jednog takta

Simulacija se završava kada su svi procesi u završnom stanju, kada se dostigne maxTicks ili kada napredak nije moguć. Poslednji slučaj se prepoznaje ako nema aktivnih ni budućih procesa, a svaki READY proces traži više resursa nego što ijedan čvor ukupno poseduje. Time se izbegava čekanje do veštačke granice za scenario koji prema tom kriterijumu ne može da napreduje.

Kvar čvora i ponovno raspoređivanje

Kvar može biti ručno izazvan preko API-ja, zakazan u scenariju ili otkriven prekoračenjem heartbeat roka. Sva tri ulaza koriste isti failNodeLocked tok. Čvor prelazi u FAILED, njegovi brojači alokacije vraćaju se na nulu, a lista procesa se obrađuje po sortiranom ID redosledu.

Svaki pogođeni proces prelazi u FAILED. Ako CanRestart vrati tačno, brojač restartovanja se uvećava, trajanje se vraća na početnu vrednost i proces prelazi u READY. Sledeće raspoređivanje može da ga smesti na drugi ONLINE čvor. Ako nema dozvoljen pokušaj, proces ostaje u završnom FAILED stanju.

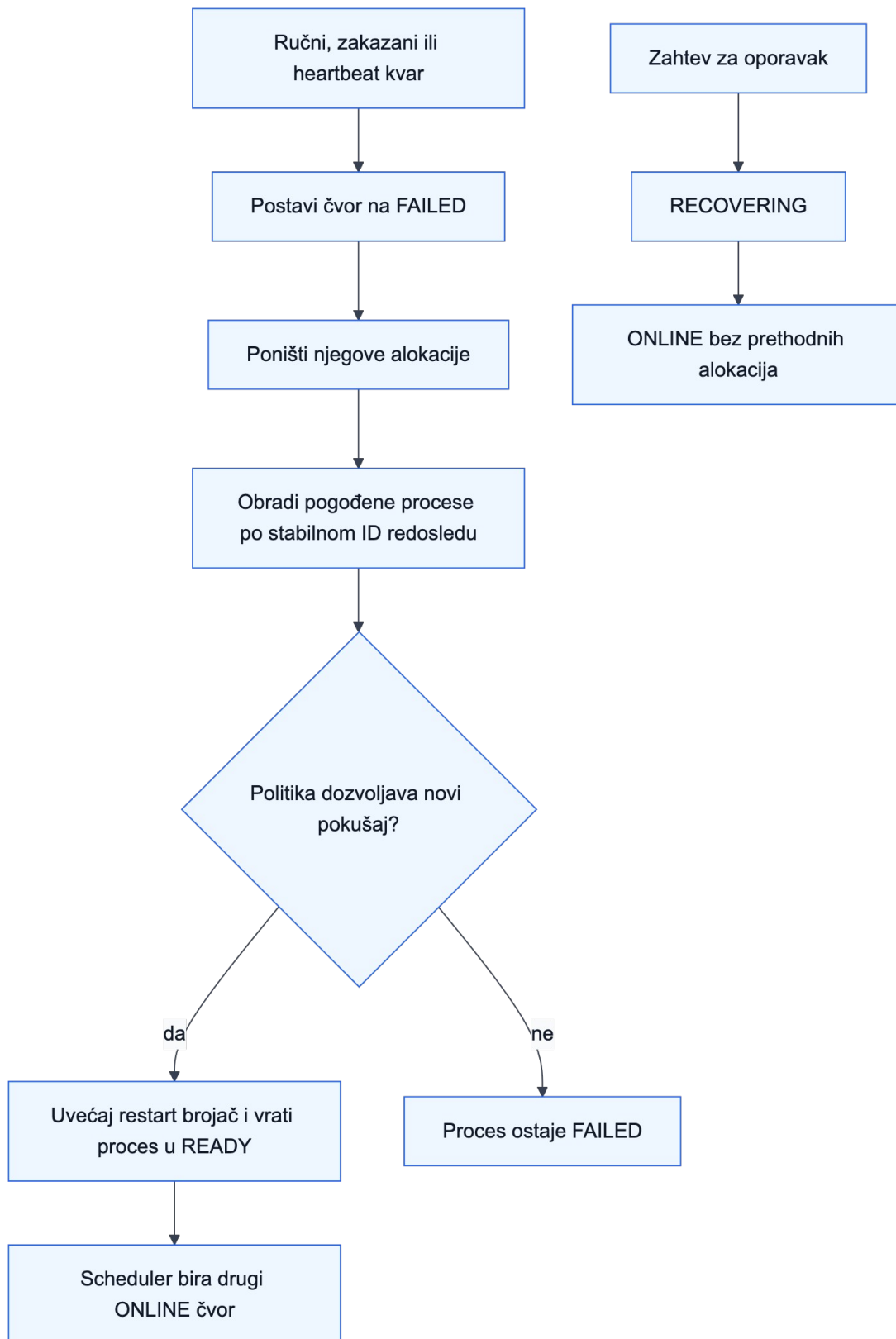
Oporavak čvora je odvojen. RecoverNode prihvata FAILED ili OFFLINE, kratko postavlja RECOVERING, zatim ONLINE i osvežava poslednji heartbeat. Ranije alokacije se ne vraćaju.

Tabela 6 navodi invarijante koje se proveravaju posle svakog uspešnog takta.

Invarijanta	Provera
Kapacitet čvora	zauzeti CPU i memorija su između nule i kapaciteta
Tačni brojači	zbir zahteva aktivnih procesa odgovara brojačima čvora
Jedinstvena alokacija	svaki RUNNING proces pripada tačno jednom čvoru
Saglasan NodeID	proces i lista čvora navode isti odnos
Bez zaostale alokacije	proces van RUNNING stanja nema NodeID
Bez nepoznatih procesa	svaki ID u listi čvora postoji u registru procesa

Ove provere se izvršavaju u runtime-u, a ne samo u testovima. Greška zato ne može tiho da ostavi prekomerno zauzet čvor ili proces koji je istovremeno raspoređen na dve lokacije.

Slika 4 prikazuje obradu kvara, odluku o restartu i naknadni oporavak čvora.



Slika 4. Obrada kvara i oporavka

REST API i komandni alati

REST servis koristi prefiks `/api/v1`. Rute su podeljene na čvorove, procese, simulaciju, raspoređivač, događaje i izveštaje. Zdravlje servisa dostupno je na `/healthz`.

Tabela 7 prikazuje glavne grupe operacija.

Resurs	Primeri operacija
Čvorovi	registracija, lista, status, kvar, oporavak, heartbeat i brisanje
Procesi	prijava, lista, pauza, nastavak, I/O čekanje, buđenje, kvar i prekid
Simulacija	start, pauza, koraci, reset, status i učitavanje scenarija
Scheduler	pregled dostupnih politika i promena aktivne politike
Izveštaji	trenutne metrike, zbirni pregled i JSON/CSV izvoz

JSON dekođer odbija nepoznata polja. Telo zahteva ograničeno je na 1 MiB. Domenske greške se prevode u HTTP statute 400, 404, 409 ili 422, dok neočekivana greška vraća 500. Server postavlja rokove za čitanje zaglavlja, čitanje tela, pisanje odgovora i idle konekcije.

`clusterctl` koristi isti API. Globalna opcija `--server` određuje adresu, a podkomande prate domenske operacije. Primer prijave čvora i procesa izgleda ovako:

```
go run ./cmd/clusterctl node add \
  --id node-1 --cpu 4 --memory 4096
```

```
go run ./cmd/clusterctl process submit \
  --id p1 --cpu 1 --memory 256 \
  --ticks 5 --priority 10
```

```
go run ./cmd/clusterctl simulation step --ticks 5
go run ./cmd/clusterctl report show
```

Samostalni simulator ne zahteva pokrenut server:

```
go run ./cmd/simulator \
  -scenario scenarios/node-failure.json \
  -format json
```

Opcija `-scheduler` menja raspoređivač iz scenarija, a `-seed` menja nenultu seed vrednost. Putanja izlaznog JSON ili CSV fajla zadaje se opcijom `-output`. Dekoder scenarija ograničava ulaz na 4 MiB i odbija nepoznata polja.

Konfiguracija i isporuka

Server čita konfiguraciju iz promenljivih okruženja. Podrazumevani port je 8080, tick traje 500 ms, heartbeat automatsko otkrivanje je isključeno vrednošću 0, a početni raspoređivač je Least Loaded. Granice su 100 čvorova i 10.000 procesa.

Tabela 8 prikazuje dostupne promenljive.

Promenljiva	Podrazumevana vrednost	Uloga
APP_PORT	8080	HTTP port
LOG_LEVEL	info	nivo JSON logova
TICK_DURATION_MS	500	real-time interval takta
HEARTBEAT_TIMEOUT_TICKS	0	broj propuštenih taktova do kvara
DEFAULT_SCHEDULER	least-loaded	početna politika
MAX_NODES	100	granica registracije čvorova
MAX_PROCESSES	10 000	granica prijave procesa
RANDOM_SEED	42	seed početnog stanja

Dockerfile koristi višefazni build. Prva faza kompajlira statički server binarni fajl, dok završna distroless slika radi kao nonroot korisnik. Docker Compose prosleđuje osnovnu konfiguraciju i mapira port 8080. GitHub Actions workflow proverava format, vet, testove sa coverage izveštajem, race detector, sve binarne fajlove i container build.

Metrike

Izveštaj se gradi iz snimka klastera. Broje se procesi po stanju, restartovanja, odlaganja raspoređivanja, kvarovi čvorova i ponovna raspoređivanja. Vremenske metrike uključuju prosečno i maksimalno čekanje, prosečno vreme obrta, propusnost i ukupan broj taktova.

CPU iskorišćenost je odnos ukupno zauzetog i ukupno dostupnog CPU kapaciteta. Memorijska iskorišćenost računa se na isti način. Obe vrednosti uzorkuju se jednom po taktu, posle raspoređivanja i pre izvršavanja procesa.

Balans opterećenja koristi standardnu devijaciju ocena čvorova. Ocena jednog čvora je ista ponderisana kombinacija koju koristi Least Loaded. Manja standardna devijacija znači da su čvorovi u proseku sličnije opterećeni. Metrika ne tvrdi da je takav raspored uvek bolji; ona meri jednu tačno definisanu osobinu.

Uspešnost je odnos TERMINATED procesa prema svim procesima. Propusnost je broj završenih procesa podeljen brojem taktova. Vreme obrta koristi razliku između završnog i prijavnog takta. Prosečno čekanje računa zbir vremena provedenog u READY stanju za procese koji su makar jednom počeli izvršavanje.

Eksperimentalni scenariji

Repozitorijum sadrži šest scenarija. Balansirano opterećenje ima dve datoteke jer se ista raspodela pokreće sa Round Robin i Least Loaded politikom. Heterogeni scenario se pokreće dva puta uz promenu opcije -scheduler. Ukupno se dobija sedam redova rezultata.

Tabela 9 opisuje ulaze.

Scenario	Čvorovi	Procesi	Scheduler	Posebna osobina
balanced.json	5	12	Round Robin	jednaki čvorovi i umereno opterećenje
balanced-least-loaded.json	5	12	Least Loaded	isti workload, drugi izbor čvora
heterogeneous.json	3	6	Least Loaded	različiti kapaciteti čvorova
overload.json	2	8	Least Loaded	više zahteva od trenutnog kapaciteta
priority-workload.json	2	7	Priority-Aware	urgentni proces dolazi u taktu 2
node-failure.json	3	5	Least Loaded	zakazan kvar čvora u taktu 4

Svi referentni ulazi koriste seed 42 i deterministički engine. Rezultati se dobijaju komandom:

```
go run ./cmd/simulator \
  -scenario scenarios/<naziv>.json \
  -format json
```

Za heterogeni scenario poređenje se ponavlja ovako:

```
go run ./cmd/simulator \
  -scenario scenarios/heterogeneous.json \
  -scheduler round-robin
```

```
go run ./cmd/simulator \
  -scenario scenarios/heterogeneous.json \
  -scheduler least-loaded
```

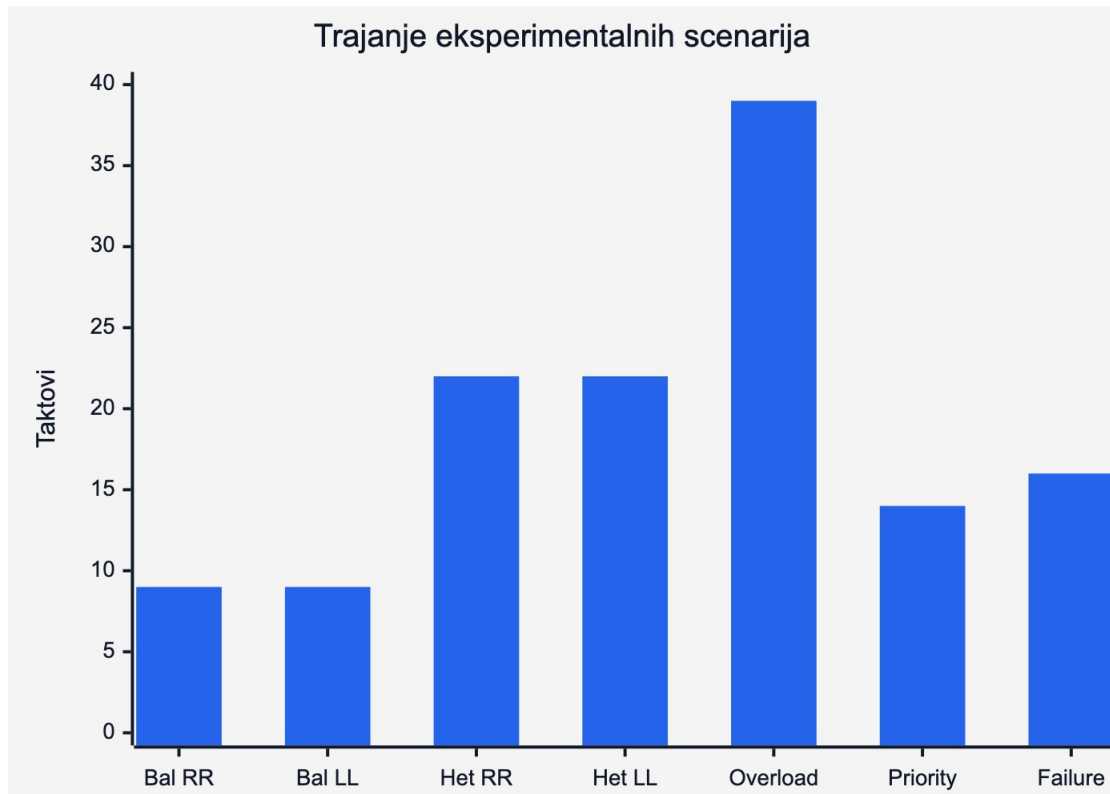
Rezultati

Tabele 10a i 10b prenose vrednosti iz verzionisanog summary.csv izveštaja. Jedan širok izveštaj podeljen je radi čitljivosti. Procenti su izračunati iz decimalnih vrednosti koje daje aplikacija.

Pokretanje	Taktovi	Čekanje	Propusnost	Uspeh
Balansirano, Round Robin	9	1,00	1,333	100%
Balansirano, Least Loaded	9	1,00	1,333	100%
Heterogeno, Round Robin	22	3,00	0,273	100%
Heterogeno, Least Loaded	22	3,00	0,273	100%
Veliko opterećenje	39	11,25	0,205	100%
Prioriteti	14	3,43	0,500	100%
Kvar čvora	16	1,80	0,313	100%

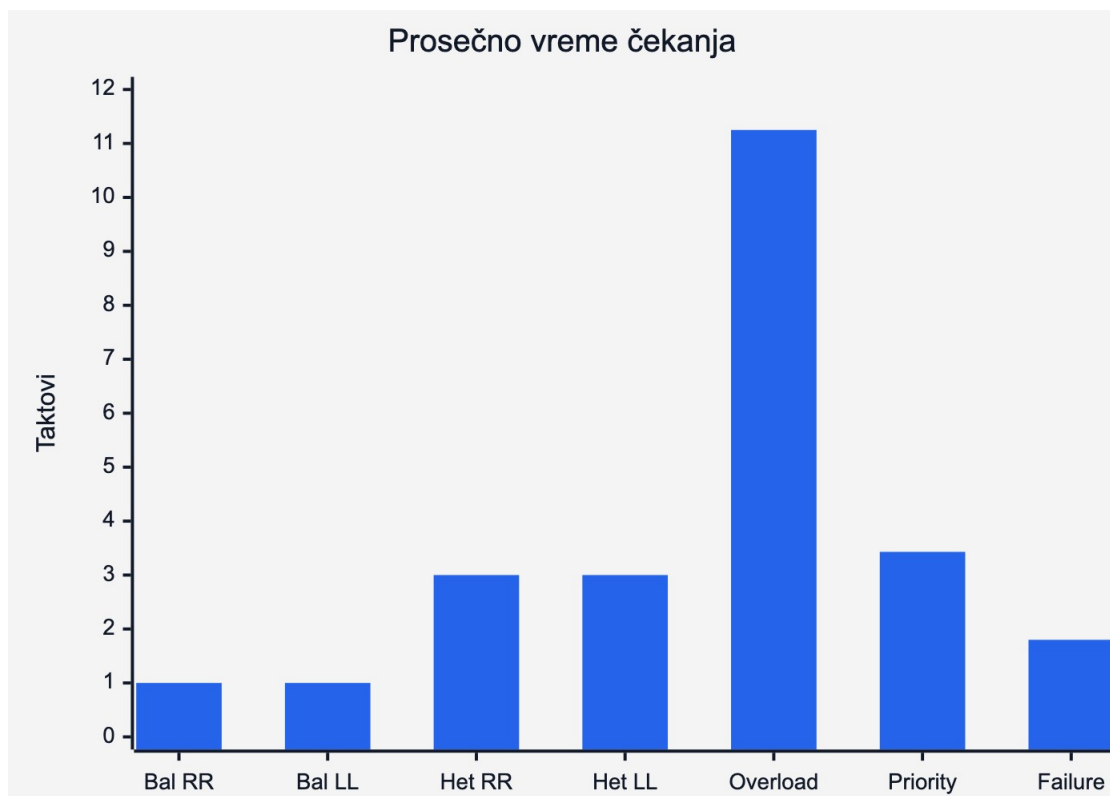
Opterećenje	Prosečan CPU	Load σ	Odlaganja	Restarti
Balansirano, Round Robin	52,22%	0,1393	0	0
Balansirano, Least Loaded	52,22%	0,0987	0	0
Heterogeno, Round Robin	49,68%	0,2417	12	0
Heterogeno, Least Loaded	49,68%	0,2452	12	0
Veliko opterećenje	89,10%	0,0962	82	0
Prioriteti	75,00%	0,0469	18	0
Kvar čvora	54,17%	0,2368	4	2

Slika 5 prikazuje ukupan broj taktova. Preopterećeni scenario traje 39 taktova, dok oba balansirana pokretanja završavaju za 9.



Slika 5. Trajanje referentnih scenarija

Slika 6 prikazuje prosečno čekanje. Rast sa 1,00 na 11,25 taktova prati ograničenje kapaciteta u preopterećenom scenariju.



Slika 6. Prosečno vreme čekanja

Tumačenje balansiranog i heterogenog opterećenja

Balansirani workload je kontrolni slučaj. Pet čvorova ima po 4 CPU jedinice i 4096 MB memorije, a 12 procesa ima iste prioritete. Round Robin i Least Loaded završavaju svih 12 procesa za 9 taktova. Prosečno čekanje je 1 takt, prosečno vreme obrta 6,17 taktova, a propusnost 1,333 procesa po taktu.

Razlika se vidi u balansu. Least Loaded smanjuje prosečnu standardnu devijaciju sa 0,1393 na 0,0987. Relativno smanjenje je približno 29%. Ukupno vreme ostaje isto jer oba rasporeda imaju dovoljno kapaciteta. Metrika ipak potvrđuje da kriterijum Least Loaded menja raspodelu, čak i kada to ne utiče na završetak.

Heterogeni scenario ima tri čvora različitih kapaciteta i šest procesa. Oba raspoređivača završavaju za 22 takta sa prosečnim čekanjem od 3 takta i 12 odlaganja. Najveće čekanje je 13 taktova. Ograničenja kapaciteta izbacuju premale čvorove za najveće procese, pa razlika u heuristici ne menja tok dovoljno da utiče na završne vremenske metrike.

Load standardna devijacija je u ovom slučaju neznatno manja kod Round Robin raspoređivača, 0,2417 naspram 0,2452. Rezultat sprečava unapred donet zaključak da Least Loaded mora pobediti u svakoj metriki. Njegova trenutna ocena gleda popunjenost pre smeštanja i ne uključuje očekivano trajanje posla. U heterogenoj raspodeli stabilan kružni izbor može slučajno dati jednako dobar ili malo bolji zbirni balans.

Tumačenje preopterećenja i prioriteta

Preopterećeni scenario ima dva čvora i osam procesa. Svi procesi završavaju, ali ne mogu svi da krenu u prvom taktu. CPU je prosečno zauzet 89,10%, memorija 79,49%, a kontroler beleži 82 neuspešna pokušaja raspoređivanja. Prosečno čekanje raste na 11,25, maksimalno na 31, a vreme obrta na 23,88 taktova.

Odlaganje nije greška. Kada nema trenutnog kapaciteta, proces ostaje READY. Završetak aktivnog procesa oslobađa resurse, pa red kasnije napreduje. Sistem ne prekoračuje kapacitet da bi veštački skratio rezultat.

Prioritetni scenario koristi dva čvora i sedam procesa. Dva procesa imaju prioritet 10, dva prioritet 5, dva prioritet 1, a proces urgent sa prioritetom 20 ulazi u taktu 2. Priority-Aware sortira red prema efektivnom prioritetu, dok aging postepeno povećava vrednost procesa koji čekaju.

Scenario završava za 14 taktova, sa prosečnim čekanjem 3,43 i 18 odlaganja. Svi procesi završavaju. Rezultat pokazuje da prioritet menja redosled, ali ne stvara dodatni kapacitet. Kada su oba čvora puna, i urgentni proces mora da sačeka oslobađanje potrebne dve CPU jedinice.

Tumačenje kvara čvora

Scenario kvara ima tri jednaka čvora sa po 2 CPU jedinice i 2048 MB memorije. Pet procesa koristi politiku ON_FAILURE i dozvoljava najviše dva restarta. U taktu 4 zakazan je kvar node-1.

Kvar pogađa dva aktivna procesa. Oba prelaze u FAILED, koriste po jedan restart pokušaj, vraćaju se u READY i kasnije dobijaju zdrave čvorove. Izveštaj zato beleži dva restarta i dva ponovna raspoređivanja. Ne beleži neuspešne završne procese.

Ukupno trajanje je 16 taktova. Prosečno čekanje iznosi 1,80, prosečno vreme obrta 11,20, a zabeležena su četiri odlaganja raspoređivanja. Propusnost je 0,3125 procesa po taktu. Završni uspeh od 100% potvrđuje da restart politika radi u granicama scenarija.

Rezultat ne predstavlja live migraciju. Pogođeni procesi počinju novi pokušaj sa punim totalTicks. Broj reschedulings označava da je proces koji je već bio pokrenut ponovo smešten. Dokument koristi izraz „ponovno raspoređivanje” da bi se ta razlika zadržala.

Ponovljivost i funkcionalna provera

Za fiksni scenario scheduler dobija stabilno sortirane podatke. Pri istom prijavnom taktu procesi se sortiraju po ID-u, a isti kriterijum važi i za čvorove. Least Loaded koristi ID kao završni kriterijum. Zakazani događaji vezani su za virtuelni takt. Zbog toga isti ulaz i ista politika daju isti izveštaj.

Ponovljivost je proverena scenarijem kvara čvora, poređenjem Round Robin i Least Loaded politika nad istim heterogenim ulazom i pozivom REST servisa kroz clusterctl. Scenario kvara daje nodeFailures: 1, restarts: 2 i reschedulings: 2, dok poređenje raspoređivača zadržava isti JSON ulaz i menja samo izabranu politiku.

Komanda korišćena za jedno od ponovljivih poređenja je:

```
go run ./cmd/simulator \
  -scenario scenarios/heterogeneous.json \
  -scheduler round-robin \
  -format csv
```

Za drugi prolaz menja se samo -scheduler least-loaded. Isto pravilo važi za seed, iako postojeći scenariji nemaju slučajnu generaciju kvarova. Seed je deo izveštaja i ostavlja prostor za buduće proširenje bez promene izlaznog ugovora.

Testiranje

Automatizovane provere obuhvataju izvršne programe i interne pakete. U završnoj proveru pokrenuto je 57 testova i podtestova. Test funkcije koriste table-driven slučajeve tamo gde ista pravila imaju više ulaza.

Tabela 11 izdvaja ponašanja koja nose najveći rizik.

Oblast	Provereno ponašanje
Domen	validacija procesa, čvora i scenarija; dozvoljeni prelazi
Scheduler	kružni kursor, preskakanje punog čvora, load ocena i aging
Kontroler	alokacija, oslobađanje, pauza, I/O, prekid i konkurentne prijave
Kvarovi	kvar čvora/procesa, restart granica, heartbeat timeout i oporavak
Scenario	transakcioni load, zakazane prijave/kvarovi, maxTicks i no-progress

Oblast	Provereno ponašanje
Engine	determinističko ponavljanje, real-time završetak, pause i close
Metrike	čekanje, obrt, propusnost, balans i JSON/CSV red
HTTP	status kodovi, validacija JSON-a, rute i scenario load
CLI	argumenti, izlaz, server konfiguracija i graceful shutdown
Store	duboke kopije i JSON round-trip snimka

Komande za punu lokalnu proveru su:

```
make fmt-check
make vet
make test
make race
make build
```

Coverage je prijavljen po paketu. Paketi sa domenskom i algoritamskom logikom nalaze se između 68,8% i 82,4% statement coverage-a u izvršenoj proveri. HTTP paket ima 45,7%, dok mali logging paket nema direktan test. Ove vrednosti se ne predstavljaju kao jedinstveni zbirni procenat jer go test ./... -cover izveštava svaki paket zasebno.

Race detector je relevantan zbog RWMutex zaštite kontrolera, mutex-a Round Robin kursora i goroutine petlje engine-a. Testovi uključuju konkurentne prijave i paralelno čitanje stanja. Prolazak race provere ne dokazuje odsustvo svake konkurentne greške, ali proverava izvršene putanje sa Go instrumentacijom.

Korektnost i složenost

Korektnost je definisana pravilima modela. Proces ne sme da bude RUNNING bez čvora. Čvor ne sme da navede proces koji ne postoji. CPU i memorijski brojači moraju odgovarati zbiru aktivnih procesa. Oslobođanje resursa mora da ukloni proces iz liste čvora. Ova pravila proverava ValidateInvariants posle svakog takta.

Neka je P broj procesa, N broj čvorova, a T broj izvršenih taktova. Kontroler u svakom taktu prolazi kroz procese da pronađe READY i RUNNING skup. Sortiranje reda košta $O(P \log P)$. Izbor čvora za svaki spreman proces u najgorem slučaju pregleda sve čvorove, pa košta $O(PN)$. Provera invarijanti ponovo prolazi kroz čvorove i njihove aktivne procese.

Gornja procena za ceo scenario je $O(T(P \log P + PN))$, uz dodatni linearni trošak provere stanja. Za zadate scenarije sa najviše 12 procesa taj trošak je mali. Procena nije zamena za merenje na konfigurisanoj granici od 10.000 procesa. Rad sa tom veličinom zahteva zaseban benchmark pre odluke o promeni reda spremnih procesa.

Memorijska složenost je $O(P + N + E)$, gde je E broj događaja. Dnevnik događaja raste tokom rada i nema runtime limit ni rotaciju. To je prihvatljivo za kratke scenarije, ali nije odgovarajuće za dugotrajan servis.

Bezbednosne i operativne granice

Server nema autentifikaciju ni autorizaciju. Svaki klijent koji može da pristupi portu može da menja stanje simulacije. Sistem zato treba pokretati lokalno ili u izolovanoj mreži. Nije predviđen za izlaganje na javni internet.

Ulazni JSON ima ograničenje veličine i odbija nepoznata polja. HTTP server koristi rokove i ograničava zaglavlja. Docker proces radi bez root privilegija. Ove mere smanjuju deo tehničkog rizika, ali ne menjaju obrazovnu namenu sistema.

Procesi su podaci. Simulator ne izvršava komandu iz polja name ili id, pa scenario ne može direktno da pokrene korisnički kod. CSV i JSON izvoz sadrže izračunate metrike, bez tajni ili podataka o autentifikaciji.

Ograničenja i dalji rad

Najveće ograničenje je apstrakcija procesa. CPU i memorija su rezervacije u modelu, ne izmerena potrošnja. Trajanje je broj taktova. Rezultat zato poredi algoritme unutar simulatora, a ne performanse stvarnog klastera.

Preempcija po vremenskom kvantu nije implementirana. Polje `timeQuantum` postoji u modelu i scenario fajlovima, ali `executeRunningLocked` umanjuje trajanje svakog aktivnog procesa u svakom taktu. Sledeća verzija može da uvede kvantni brojač, `PROCESS_PREEMPTED` događaj i povratak u `READY`.

Restart ne koristi checkpoint. Kvar vraća `remainingTicks` na `totalTicks`. Za model nastavka bilo bi potrebno čuvati napredak i definisati da li se deo rada ponavlja. Za stvarni proces bili bi potrebni memorijska slika, otvoreni resursi i protokol oporavka.

Heartbeat je izražen u virtuelnim taktovima. Ne modeluje kašnjenje mreže, privremenu particiju ili lažno pozitivan kvar. Sistem ima jedan kontroler i nema konsenzus između replika. Distribuirani log i izbor vođe predstavljaju zaseban problem, kakav obrađuje Raft literatura (Ongaro i Ousterhout 2014.).

Least Loaded koristi jednake težine CPU-a i memorije. Konfigurabilne težine i ocena posle zamišljenog smeštanja procesa dale bi precizniji kriterijum. Scheduler takođe ne poznaje oznake čvorova, afinitet, zone ili posebne uređaje. Kubernetes dokumentacija pokazuje kako oznake i pravila utiču na izbor čvora (Kubernetes Documentation 2026.).

Snapshot pomoćne funkcije nisu trajno skladište servera. Povezivanje sa fajlom ili bazom zahtevalo bi verzionisanje formata, proveru invarijanti pri učitavanju i odluku kada se snimak pravi. Trenutni `ReadSnapshot` proverava JSON oblik, ali ne izvršava punu domensku validaciju obnovljenog klastera.

Eksperimentalni skup može da se proširi. Korisni naredni slučajevi su postepeni gubitak kapaciteta, više kvarova, procesni kvar bez kvara čvora i mešovita CPU/memorijska opterećenja. Kod scenarija sa slučajno generisanim događajima rezultat treba ponoviti nad više seed vrednosti i prijaviti srednju vrednost, standardnu devijaciju i interval pouzdanosti.

Zaključak

Razvijena je Go aplikacija koja upravljanje procesima u klasteru prikazuje kroz konkretan i proverljiv model. Sistem ima osam stanja procesa, pet statusa čvora, tri raspoređivača, deterministički engine, REST API, komandni klijent i izvoz metrika. Kvarovi čvora i procesa koriste isti kontrolni tok, a resursne invarijante se proveravaju posle svakog takta.

Rezultati pokazuju razliku između heuristike i kapaciteta. Least Loaded je u balansiranom scenariju smanjio odstupanje opterećenja za približno 29%, bez promene ukupnog vremena. U heterogenom scenariju oba poređena raspoređivača dala su iste vremenske metrike jer su ograničenja kapaciteta dominirala izborom. Preopterećenje je povećalo prosečno čekanje na 11,25 taktova, dok je CPU ostao zauzet 89,10%.

Scenario kvara potvrđuje ograničeni restart i ponovno raspoređivanje. Jedan kvar čvora izazvao je dva restarta i dva nova smeštanja, a svih pet procesa završilo je za 16 taktova. To nije live migracija. Svaki pogođeni proces počeo je novi pokušaj od punog trajanja, što odgovara kodu i prijavljenim metrikama.

Simulacija ostavlja jasne granice. Nema stvarnih udaljenih procesa, preempcije po kvantu, trajne baze, autentifikacije ni distribuiranog konsenzusa. Te granice omogućavaju da odluka schedulera, promena stanja i posledica kvara ostanu vidljive u kodu, testovima i ponovljivim scenarijima.

Spisak literature

Ajmani, Sameer. 2014. *Go Concurrency Patterns: Context*. <https://go.dev/blog/context>.

Kubernetes Documentation. 2025. *Scheduling Framework*.
<https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/>.

Kubernetes Documentation. 2026. *Assigning Pods to Nodes*.
<https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/>.

Ongaro, Diego, i John Ousterhout. 2014. „In Search of an Understandable Consensus Algorithm“. *2014 USENIX Annual Technical Conference*, 305–19.
<https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.

Silberschatz, Abraham, Peter B. Galvin, i Greg Gagne. 2014. *Operating System Concepts Essentials*, 2nd edition. Wiley.

Tanenbaum, Andrew S., i Herbert Bos. 2015. *Modern Operating Systems*, 4th edition. Pearson.